Python Programming — Database Module

Python & SQLite3

Storing and retrieving data with Python's built-in database

1 Connect & Create

2 Insert Data

3 Query Data

4 Update & Delete

What is SQLite3?

A lightweight, file-based database — no separate server needed

Key Characteristics

- SQLite stores an entire database in a single file on disk (e.g. college.db).
- No installation, no server process, no username/password — perfect for learning and small apps.
- Python ships with the sqlite3 module built-in — nothing extra to install.
- Uses standard SQL: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE.

Why beginners start here

- No setup hassle — just import sqlite3 and go.
- Great for learning how any real database works (MySQL, PostgreSQL use the same SQL ideas).
- Ideal for college projects, prototypes, and small desktop apps.

The SQLite3 Workflow — 5 Steps

Every Python + SQLite program follows this same pattern

1

Import & Connect

```
import sqlite3
conn =
sqlite3.connect(
  'college.db')
```

2

Create a Cursor

```
cursor =
conn.cursor()
```

3

Execute SQL

```
cursor.execute(
  'SELECT * FROM
t')
```

4

Commit Changes

```
conn.commit()
```

5

Close Connection

```
conn.close()
```

Step 1 & 2: Connect and Create a Table

We'll build a small students database, table by table

```
import sqlite3

# Connect (creates the file if it doesn't exist)
conn = sqlite3.connect('college.db')
cursor = conn.cursor()

cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        roll_no INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        branch TEXT,
        cgpa REAL
    )
''')

conn.commit()
print('Table created successfully!')
```

Output

```
Table created successfully!
```

Notes

- roll_no as PRIMARY KEY uniquely identifies each row.
- IF NOT EXISTS avoids an error if you re-run the script.
- conn.commit() saves the change permanently to the file.

Step 3: Inserting Data

Add one row at a time, or many rows at once

```
# Insert a single student
cursor.execute(
    'INSERT INTO students VALUES (?, ?, ?, ?)',
    (101, 'Aditi Sharma', 'CSE', 8.7)
)

# Insert many students at once
students_data = [
    (102, 'Rahul Verma', 'CSE', 7.9),
    (103, 'Sneha Iyer', 'ECE', 9.1),
    (104, 'Karthik Raj', 'CSE', 8.3),
]
cursor.executemany(
    'INSERT INTO students VALUES (?, ?, ?, ?)',
    students_data
)

conn.commit()
print('Data inserted successfully!')
```

Output

```
Data inserted successfully!
```

Why the ? placeholders?

The ? marks are placeholders — Python safely inserts the values for you. This protects against SQL injection, unlike pasting values directly into the SQL string.

execute() → one row

executemany() → a list of rows

Step 4: Reading Data with SELECT

fetchall() retrieves every matching row as a list of tuples

```
cursor.execute('SELECT * FROM students')
rows = cursor.fetchall()

for row in rows:
    print(row)
```

Output

```
(101, 'Aditi Sharma', 'CSE', 8.7)
(102, 'Rahul Verma', 'CSE', 7.9)
(103, 'Sneha Iyer', 'ECE', 9.1)
(104, 'Karthik Raj', 'CSE', 8.3)
```

Three ways to fetch results

`fetchone()`

Returns the next single row, or None if no more rows

`fetchmany(n)`

Returns the next n rows as a list

`fetchall()`

Returns every remaining row as a list of tuples

Filtering with WHERE — Safely

Always pass values as parameters, never build SQL with string formatting

```
branch = 'CSE'
cursor.execute(
    'SELECT name, cgpa FROM students WHERE branch = ?',
    (branch,)
)

for name, cgpa in cursor.fetchall():
    print(f'{name}: {cgpa} CGPA')
```

Output

```
Aditi Sharma: 8.7 CGPA
Rahul Verma: 7.9 CGPA
Karthik Raj: 8.3 CGPA
```

Avoid this — SQL Injection Risk

```
cursor.execute(
  f"SELECT * FROM students "
  f"WHERE branch = '{branch}'")
```

Do this instead

Pass values as a tuple with ? placeholders. SQLite escapes them safely, so user input can never break out of the query.

Step 5: Updating and Deleting Records

Modify or remove rows — always target them with WHERE

```
# Update Rahul's CGPA
cursor.execute(
    'UPDATE students SET cgpa = ? WHERE roll_no = ?',
    (9.0, 102)
)

# Remove Karthik's record
cursor.execute(
    'DELETE FROM students WHERE roll_no = ?',
    (104,)
)

conn.commit()
print('Update and delete done')
```

Output

```
Update and delete done
```

Careful!

Leaving out the WHERE clause updates or deletes EVERY row in the table — always double-check it before running.

Closing the Connection Safely

Wrap database code in try / except / finally

```
try:
    conn = sqlite3.connect('college.db')
    cursor = conn.cursor()
    cursor.execute('SELECT COUNT(*) FROM students')
    print('Total students:', cursor.fetchone()[0])

except sqlite3.Error as e:
    print('Database error:', e)

finally:
    conn.close()
```

Output

```
Total students: 3
```

Why this pattern?

- try: runs the risky database code.
- except sqlite3.Error: catches problems (bad SQL, locked file, etc.) instead of crashing.
- finally: always closes the connection — even if an error happened. An open connection can lock the database file.

Creation of Database and Connect

```
import sqlite3
conn = sqlite3.connect("college.db")
print("Database Connected")
```

Creation of Table under Database

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS student(
id INTEGER PRIMARY KEY,
name TEXT,
department TEXT
)
""")
conn.commit()
print("Table Created")
```

Creation of Table under Database

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS student(
id INTEGER PRIMARY KEY,
name TEXT,
department TEXT
)
""")
conn.commit()
print("Table Created")
```

Insert Data [Row]

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute(
"INSERT INTO student VALUES(5,'Alpha','CSE')"
)
conn.commit()
print("Record Inserted")
```

Read Data

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM student")
rows = cursor.fetchall()

for row in rows:
     print(row)
```

Insert Multiple Rows of Data

```
import sqlite3

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

students = [

(6,'Rajesh','IT'),

(7,'Vinay','ECE'),

(8,'Beta','AIDS')

]

cursor.executemany(

"INSERT INTO student VALUES(?,?,?)",

students

)

conn.commit()

#The ? symbols are placeholders. They tell SQLite: "The actual

values will be supplied separately."
```

Update Record

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute(
"UPDATE student SET name='Gamma' WHERE id=4"
)
conn.commit()
```

Delete Record

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()


cursor.execute(
"DELETE FROM student WHERE id=4"
)


conn.commit()
```

Common Beginner Mistakes

Watch out for these while practicing

Forgetting conn.commit()

Without it, INSERT/UPDATE/DELETE changes are lost when the connection closes.

Not closing the connection

Leaves the database file locked — the next run may fail to connect.

Building SQL with f-strings

f"WHERE name = '{name}'" opens the door to SQL injection — always use ? placeholders.

Wrong number of placeholders

The tuple of values must match the number of ? marks exactly, or Python raises an error.

Key Takeaways

1. sqlite3 is Python's built-in, file-based database — no server or setup needed.
2. Workflow: connect() → cursor() → execute() → commit() → close().
3. Use execute() for one row, executemany() for many rows at once.
4. Always fetch results with fetchone(), fetchmany(n), or fetchall().
5. Always use ? placeholders for values — never build SQL with string formatting.
6. Wrap database code in try / except / finally, and always close the connection.